PROJECT ATHENA

Machine Learning, Intelligence & Training Architecture

Implementation-oriented reference for ATHENA v2.7

Purpose. This document explains the intended role of machine learning in ATHENA: what ML should learn, what it must not control, how training and validation work, how probabilities become trustworthy, how ML interacts with AMEF, how drift and failures are handled, and how the system progresses from cold start to production authority.

1. Executive Summary

ATHENA was not designed around the idea of “put an ML model in front of a trading API and let it predict the next candle.” The intended architecture separates an **intelligence plane** from a **deterministic execution plane**. Deterministic rules remain responsible for hard constraints, risk, execution safety and governance. ML provides probabilistic evidence about market state, setup quality, regime, and expected outcomes. AMEF provides historical experience and contextual evidence. The decision layer combines these sources without allowing any single source—including ML or memory—to become an unchecked authority.

Core idea

`Market data → features → regime/context → ML inference → calibrated probability → AMEF retrieval → evidence fusion → governed decision → deterministic risk/execution → outcome attribution → training feedback.`

2. What ML Is Supposed to Do

Role	What ML contributes
Regime intelligence	Estimate market conditions such as trend, range, volatility, momentum and transition states; the deterministic baseline remains the initial authority.
Setup quality	Estimate the probability that a technically valid setup will reach its intended outcome under the current context.
Trade outcome probability	Produce calibrated probabilities rather than binary BUY/SELL commands.
Context interaction	Learn nonlinear relationships between features that rules alone may miss—for example interactions among volatility, momentum, structure and session conditions.
Signal filtering	Help distinguish technically valid but historically weak setups from stronger setups.
Risk intelligence	Support evidence about expected adverse/favorable excursion, stop/target suitability and setup quality; hard risk limits remain deterministic.
Anomaly / uncertainty awareness	Detect situations where the current state differs from the training distribution or where model confidence is unreliable.
Adaptive intelligence	Improve slowly through governed retraining and challenger models rather than changing the live strategy after every trade.

3. What ML Is NOT Allowed to Do

• It is not the sole source of truth for whether a trade is legally/executionally possible.

• It does not override maximum risk, exposure, spread, market-hours, broker or execution constraints.

• It should not directly send orders simply because it predicts a high probability.

• It does not replace deterministic technical calculations such as MACD/RSI, price structure, POI lifecycle rules, or risk calculations.

• It does not treat similarity to a historical memory as proof that the same outcome will happen again.

• It does not continuously rewrite itself from individual live trades.

4. ATHENA Intelligence Planes

The architectural separation is fundamental. The intelligence plane can be sophisticated, probabilistic and experimental. The execution plane must remain predictable, bounded and auditable.

```
INTELLIGENCE PLANE
Market features → ML → calibration → AMEF retrieval → evidence scoring → decision recommendation

GOVERNANCE / DECISION PLANE
Strategy rules → confidence/authority → portfolio governor → risk controls → conflict resolution

EXECUTION PLANE
Broker checks → order construction → SL/TP → execution → monitoring → reconciliation
```

5. The ML Data Pipeline

Stage	Purpose
Raw ingestion	Collect OHLCV/ticks/spread/session and other approved market/context data.
Cleaning	Remove or flag corrupted, duplicated, impossible or incomplete observations.
Alignment	Synchronize timeframes and prevent accidental future information from entering a sample.
Feature engineering	Convert market state into measurable inputs: returns, volatility, momentum, structure, distances to POIs, spread, session and other governed features.
Label generation	Define what the model is actually learning using future outcome windows or other explicit targets.
Dataset versioning	Record feature definitions, label definitions, data windows and code/model versions.
Training	Fit a candidate model only on permitted historical data.
Validation	Use time-ordered validation rather than random shuffling of time-series observations.
Calibration	Convert raw model scores into probabilities that correspond reasonably to observed frequencies.
Promotion	Only a candidate that passes governance, performance, calibration and stability gates can become a challenger or production model.

6. Training: The Whole Idea

Training should answer a clearly defined question. ATHENA should not train a vague model to “predict price.” A useful target is conditional and decision-related, for example: *given this market state and a candidate setup, what is the probability of achieving the strategy-defined outcome before the invalidation condition?*

Example training sample

At time t: EURUSD, H1/M15/M5 context, trend/structure state, MACD state, volatility, spread, session, news-state features, POI properties and strategy metadata are captured. The label is generated only from information after t—for example whether TP was reached before SL within a defined horizon. The sample must never contain information that would not have been known at decision time.

Training objectives should evolve in stages

• **Stage 1:** deterministic baseline and data/label integrity. Prove that the pipeline works before relying on ML.

• **Stage 2:** train an ML model in shadow mode. It observes live conditions but cannot influence execution.

• **Stage 3:** advisory mode. ML contributes evidence to decisions while deterministic logic retains authority.

• **Stage 4:** executive mode. ML can influence decisions within explicit authority bounds after calibration and stability requirements are met.

• **Stage 5:** production model lifecycle with monitoring, retraining, challenger models, rollback and auditability.

7. Time-Series Validation & Leakage Prevention

Financial time series are not ordinary IID datasets. ATHENA therefore requires time-ordered validation. A model should train on the past and validate on later periods. Randomly shuffling candles can leak regime/context information and create unrealistically optimistic results.

```
A robust evaluation structure can be thought of as:
TRAIN → VALIDATE → TEST → FORWARD / WALK-FORWARD → LIVE SHADOW
```

• Use a strict information cutoff for every feature and label.

• Be especially careful with rolling indicators, higher-timeframe aggregation, news timestamps and future-bar labels.

• Keep a final untouched test period for honest evaluation.

• Prefer walk-forward evaluation for strategy-facing models.

• Evaluate by regime, symbol, session and market condition—not only by one aggregate score.

8. Calibration: Turning Scores Into Trustworthy Probabilities

A model output of 0.80 should not automatically be interpreted as an 80% chance of success. Calibration asks whether predicted probabilities match observed frequencies. ATHENA therefore treats calibration as a governance layer, not cosmetic post-processing.

Example: among sufficiently similar cases where ATHENA predicts approximately 0.70, roughly 70% should achieve the defined target under the same label definition. If that relationship breaks, the model's authority should be reduced or disabled.

• Calibration must use time-ordered data.

• There should be minimum sample floors before trusting a calibration estimate.

• Fallback behavior is required when calibration data are insufficient or unstable.

• Calibration quality should be monitored separately from raw predictive performance.

9. ML + AMEF: Complementary, Not Identical

AMEF is ATHENA's governed experience fabric. ML learns statistical relationships from datasets; AMEF retrieves relevant historical experience and contextual evidence. They solve related but different problems.

```
CURRENT STATE → ML probability
CURRENT STATE → AMEF query → similar historical memories → memory evidence
Then the decision layer evaluates both alongside deterministic strategy evidence.
```

Memory authority should depend on factors such as sample sufficiency, effective sample size, memory quality, regime compatibility, calibration/reliability, recency and stability. A memory retrieval score must therefore not simply be added to an ML probability as though both were the same mathematical quantity.

10. Regime Classification

Regime classification is one of the most important potential ML roles because the same setup can behave differently in different environments. ATHENA's baseline should begin with a deterministic regime classifier, then introduce advanced probabilistic regime models once the data and governance infrastructure are ready.

• Trend vs range

• Low vs high volatility

• Momentum expansion vs contraction

• Stable regime vs transition

• Potential anomalous / out-of-distribution state

11. Drift Handling

A model can remain technically functional while becoming less useful because the market distribution changes. ATHENA therefore needs explicit drift governance rather than assuming retraining automatically fixes everything.

```
NORMAL → WATCH → DEGRADED → SHADOW / REVALIDATION → RECOVERED
■ FAILSAFE / DISABLE
```

• Monitor feature distribution changes.

• Monitor prediction distribution changes.

• Monitor calibration deterioration.

• Monitor realized performance against expected performance.

• Compare performance by regime and symbol.

• Do not automatically promote a newly retrained model simply because it performs better on one backtest.

12. Challenger Models

ATHENA should be able to train a candidate model without replacing the live model immediately. A challenger can run in parallel, producing predictions for comparison. Promotion should require predefined gates around predictive performance, calibration, stability, data coverage, regime behavior and operational health.

```
Champion model → live authority
Challenger model → shadow predictions → evaluation → approval → promotion or rejection
```

13. Online Learning vs Governed Retraining

ATHENA should not interpret every new trade as permission to alter model weights. Fast adaptation belongs primarily in governed memory/context mechanisms; durable model changes should be slower and evidence-based. This resolves the v2.6 concern about the tension between rapid memory adaptation and slow model governance.

Fast loop: new observation → AMEF memory/feedback → retrieval behavior can adapt within governance limits.
Slow loop: accumulated dataset → scheduled/triggered retraining → validation → calibration → challenger → promotion.

14. What ATHENA Should Train

Candidate model	Primary use
Regime model	Estimate market regime or transition probabilities.
Setup-quality model	Estimate whether a strategy-qualified setup has favorable historical conditional behavior.
Outcome model	Estimate probability of target-before-invalidation under a defined horizon.
Risk/MAE model	Estimate adverse excursion or setup-specific risk characteristics where sufficiently supported by data.
Anomaly/OOD model	Identify market states unlike the model's reliable training distribution.

15. Model Evaluation

ATHENA should not optimize around accuracy alone. A classifier can have high accuracy while being useless for trading if class imbalance, calibration, costs and regime behavior are ignored.

• Precision / recall or equivalent classification metrics where appropriate.

• ROC-AUC / PR-AUC where meaningful.

• Brier score and calibration error for probabilistic outputs.

• Performance by probability bucket.

• Performance by regime, symbol and session.

• Economic metrics from a strategy-level evaluation: expectancy, drawdown, profit factor, hit rate and risk-adjusted outcomes.

• Stability across walk-forward periods.

• Comparison against deterministic baseline and simple benchmark models.

16. The ML Decision Lifecycle in ATHENA

A representative live decision should look like this:

```
1. Market data arrives
2. Data quality gate checks the observation
3. Features are constructed using only available information
4. Deterministic indicators/structure establish baseline context
5. Regime is classified
6. ML models produce predictions
7. Predictions are calibrated and checked for validity
8. AMEF retrieves relevant historical experience
9. Memory and ML evidence are scored independently
10. Strategy orchestrator evaluates the combined evidence
11. Portfolio/risk governor applies hard constraints
12. Decision becomes BUY / SELL / WAIT / REJECT
13. Executioner performs broker-safe execution if authorized
14. Trade is monitored and reconciled
15. Outcome is attributed back to the decision, model, regime and memories
```

17. Attribution: How ATHENA Learns What Actually Worked

Attribution is essential. If a trade wins, ATHENA should not automatically conclude that ML was correct. The outcome may have resulted from deterministic structure, news, memory, execution quality or chance. ATHENA should record the evidence available at decision time and later compare it with the realized outcome.

• Decision attribution: what evidence supported the action?

• Model attribution: what did each model predict?

• Regime attribution: what regime was active?

• Memory attribution: which memories were retrieved and how relevant were they?

• Execution attribution: did slippage/spread/fill quality alter the outcome?

• Counterfactual attribution: what would have happened under rejected alternatives, where this can be measured without contaminating the live decision?

18. Counterfactuals

Counterfactual analysis belongs in the learning/evaluation layer, not in a way that leaks future information into a live decision. Afterward, ATHENA can study rejected setups, alternative entry/exit policies or different model recommendations to understand whether the decision process was robust.

19. Cold Start

A new ATHENA installation has no trustworthy personal experience. Therefore **cold-start authority is zero** for AMEF-derived learning claims. The system should rely on validated deterministic logic and properly trained/calibrated models. As sufficient experience accumulates, memory can gradually become advisory and then gain authority under explicit governance rules.

20. Authority Ladder

The intended deployment ladder is:

OFF → SHADOW → ADVISORY → EXECUTIVE → PRODUCTION

OFF: no influence.
SHADOW: predictions logged only.

ADVISORY: predictions inform the decision but cannot override deterministic gates.
EXECUTIVE: model evidence can materially affect decisions within bounded authority.
PRODUCTION: approved model operates under continuous monitoring, rollback and governance.

21. Model Registry & Lineage

Every production-capable model should be identifiable and reproducible. ATHENA should record at least:

• Model ID and version

• Training dataset/version and time range

• Feature schema/version

• Label definition/version

• Training code/version

• Hyperparameters

• Validation/test periods

• Calibration method/version

• Performance and calibration metrics

• Supported symbols/regimes

• Promotion/approval status

• Creation, promotion and retirement timestamps

• Hashes or other lineage identifiers for reproducibility

22. Failure & Degraded Modes

The ML layer must fail safely. If an embedding service, model, calibration layer, feature calculation, data source or model registry is unavailable, ATHENA should not invent confidence.

• Model unavailable → fall back to deterministic strategy behavior if permitted.

• Calibration invalid → reduce ML authority or disable ML contribution.

• Drift detected → move model to degraded/shadow state.

• Insufficient data → use baseline/fallback rather than extrapolate.

• Feature mismatch → reject inference rather than silently continue.

• AMEF unavailable → continue with non-memory evidence if the strategy allows it.

• Execution/risk systems remain authoritative regardless of ML availability.

23. PostgreSQL + pgvector + ML Architecture

With the current migration to Supabase PostgreSQL, ATHENA can keep structured operational data and vector memory in the same canonical persistence layer. pgvector should serve the vector-retrieval portion of AMEF; it is not itself the AMEF intelligence layer.

```
Supabase PostgreSQL
■■■ Trades / positions / execution records
■■■ Market data / news / sentiment
■■■ Strategy & orchestration state
■■■ ML model registry / metrics / calibration records
■■■ Training dataset metadata & lineage
■■■ AMEF memories + pgvector embeddings

ML service
■■■ feature pipeline
■■■ inference
■■■ calibration
```

```
■■■ drift monitoring
■■■ training/evaluation jobs
```

24. Recommended Development Order for ATHENA v2.7

1. Lock down deterministic baseline and data contracts.
2. Build feature pipeline with strict timestamp/leakage controls.
3. Define exact labels and outcome horizons.
4. Build reproducible training/evaluation pipeline.
5. Train simple baseline models before complex models.
6. Add time-series walk-forward validation.
7. Add calibration and sample-floor governance.
8. Add inference service and model registry.
9. Run ML in shadow mode.
10. Integrate AMEF retrieval and independent memory scoring.
11. Add attribution and feedback logging.
12. Add drift state machine and degraded modes.
13. Add challenger models and controlled promotion.
14. Gradually increase authority only after evidence supports it.

25. Final Mental Model

The cleanest way to think about ATHENA's ML is:

Rules define what is allowed.
Technical/news/context engines describe what is happening.
ML estimates what is likely.
AMEF remembers what ATHENA has experienced.
Calibration tells ATHENA how much to trust probability estimates.
Drift monitoring tells ATHENA when that trust is deteriorating.
Attribution tells ATHENA what contributed to the outcome.
Training improves the models slowly and deliberately.
The risk/execution plane retains hard authority.

In short: ATHENA's ML was never meant to be a magic predictor. It is a governed probabilistic intelligence layer inside a larger decision system.